🚧 L16: Conditional Drawing

The Security Fence - Boundary Detection

Neon City Cyber Academy

Module 4: Visual Algorithms

👥 Your Team

Emily Chen - *Handler*

> "Drones can't fly forever. You need boundaries. You need collision detection. You need IF statements."

Cole Martinez - *Tech Lead*

> "Every game has a play area. Every robot has safe zones. This is how we code 'Do NOT cross this line.'"

🎯 The Problem: Infinite Boundaries

Without boundaries:

- Drones fly off-screen
- Game characters walk through walls
- Robots crash into obstacles

Solution: Conditional logic to CHECK position!

🗺️ The Security Fence

SAFE OUT OF BOUNDS

Boundaries: X(100-400), Y(100-300)

📏 Boundary Checking Code

```
# Safe zone boundaries
min_x, max_x = 100, 400
min_y, max_y = 100, 300

# Drone position
x, y = 250, 200

# Check if inside safe zone
if min_x <= x <= max_x and min_y <= y <= max_y:
    print("Position valid - SAFE")
else:
    print("WARNING: Out of bounds!")
```

Output: `Position valid - SAFE`

💬 Emily's Boundary Strategy:

> "Four conditions to check safety:"

1. **Left boundary:** `x >= min_x`
2. **Right boundary:** `x <= max_x`
3. **Top boundary:** `y >= min_y`
4. **Bottom boundary:** `y <= max_y`

> "All FOUR must be True = safe. ANY one False = danger!"

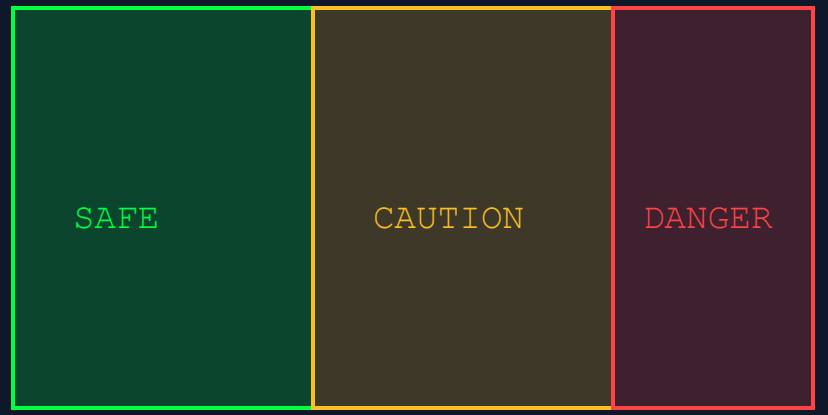
🎮 Game Collision Detection

```
# Player position
player_x = 150

# Wall position
wall_x = 200

# Check collision
if player_x >= wall_x:
    print("COLLISION! Player hit wall")
else:
    distance_to_wall = wall_x - player_x
    print(f"Safe: {distance_to_wall} units from wall")
```

🚦 Multiple Zones



🎯 Multi-Zone Code

```
x = 275  # Position

if x < 200:
    zone = "SAFE"
    color = "green"
elif x < 350:
    zone = "CAUTION"
    color = "yellow"
else:
    zone = "DANGER"
    color = "red"

print(f"Zone: {zone} ({color})")
```

Output: `Zone: CAUTION (yellow)`

💬 Cole's Collision Formula:

> "Distance between objects:"

```
# Object A at x1, Object B at x2
distance = abs(x2 - x1)

# Collision if distance < sum of radii
radius_a = 10
radius_b = 15

if distance < (radius_a + radius_b):
    print("COLLISION!")
```

> "abs() ensures positive distance - direction doesn't matter!"

🔲 Rectangle Collision (AABB)

Axis-Aligned Bounding Box:

```
# Rectangle 1
r1_x, r1_y = 100, 100
r1_width, r1_height = 50, 50

# Rectangle 2
r2_x, r2_y = 120, 120
r2_width, r2_height = 50, 50

# Check overlap
overlap_x = r1_x < r2_x + r2_width and r1_x + r1_width > r2_x
overlap_y = r1_y < r2_y + r2_height and r1_y + r1_height > r2_y

if overlap_x and overlap_y:
    print("Rectangles overlap!")
```

🎯 Clamping Values

Keep value WITHIN bounds:

```
# Clamp position to safe zone
min_x, max_x = 0, 800

x = 950  # Too far right!

# Clamp to maximum
if x > max_x:
    x = max_x

print(f"Clamped position: {x}")  # 800
```

Used in: Camera movement, player bounds, UI constraints

💬 Emily's Clamp Function:

```
def clamp(value, min_val, max_val):
    if value < min_val:
        return min_val
    elif value > max_val:
        return max_val
    else:
        return value

# Test
print(clamp(150, 0, 100))    # 100 (clamped to max)
print(clamp(-50, 0, 100))    # 0 (clamped to min)
print(clamp(50, 0, 100))     # 50 (unchanged)
```

✅ Lesson Complete!

You Mastered:

- ✅ Boundary checking with conditionals
- ✅ Multi-zone detection (safe/caution/danger)
- ✅ Collision detection algorithms
- ✅ Clamping values to valid ranges
- ✅ Real-world applications in games/robotics

Next: L17 - Project: Grid Defense

Build a procedural laser grid system

🏆 Your Assignment

Create a **Boundary Validator:**

1. Define safe zone: X(0-500), Y(0-300)
2. Check 5 test coordinates
3. Print "SAFE" or "OUT OF BOUNDS" for each
4. Count how many are safe